

Trabalho - Sistemas Operacionais I

Igor dos Reis Gomes

RA: 241025265

Fernando Hiroshi Murusaki

RA: 241025851

Murilo Tomaz Gonzaga

RA: 241024684

Lista de Exercícios 1 - Introdução aos Sistemas Operacionais

1- a) O contador de programas é um registrador que se encontra dentro da UCP e tem como função armazenar o endereço de memória da próxima instrução que será executada pela UCP.

b) É um registrador da UCP que tem como função armazenar o endereço da memória do topo atual da pilha, uma estrutura de dados temporárias usadas para gerenciar todas chamadas de função de um programa.

c) O PSW também é um registrador que está presente dentro da UCP, o PSW possui flags que realizam o controle da UCP. Um exemplo de flag de controle é o de modo da UCP, sendo 0 o modo núcleo e 1 o modo usuário.

2- Os controladores de dispositivos ^{são} processadores especializados em gerenciar a troca de dados dos dispositivos de E/S, para não sobrecarregar a UCP. Os controladores são programáveis (drivers).

3- Interrupções da UCP são mecanismos de interação entre os canais autônomos de E/S e a UCP. Foram criadas para que fosse possível que as operações de E/S ocorressem em paralelo com o processamento de outras atividades da UCP. Os principais tipos de interrupção são a de hardware e a de software.

As interrupções de hardware são sinais gerados por dispositivos externos da UCP, tendo como função notificar a UCP sobre a ocorrência de um evento. Já as interrupções de software são geradas pela própria UCP. Possuem 2 funções: fazer chamadas de sistema e realizar o tratamento de operações ilegais ou execução de uma instrução malformada.

4- As duas principais funções do SO são: estender o hardware, ou seja, oferece uma interface mais amigável do hardware para o usuário e a segunda função é realizar o gerenciamento dos elementos do computador, isto é, permite o uso eficiente e controlado do hardware e permite o uso compartilhado e protegido dos dispositivos de hardware.

5- Multiprogramação é uma técnica que se baseia em dividir a memória do computador em várias partes, para que assim se torne possível armazenar na memória várias tarefas ao mesmo tempo, assim a UCP pode se manter ocupada por quase todo o tempo. Já o time sharing é uma variação da multiprogramação, na qual cada usuário tem seu terminal e a UCP é dividida entre os usuários. O primeiro sistema foi o CTSS.

6- Não, esses dois conceitos foram criados em tempos diferentes. Os sistemas de tempo compartilhado foram criados posteriormente, uma vez que esse tipo de sistema também deve ser multiprogramado. Esses dois conceitos têm objetivos diferentes.

7- A técnica de Spooling consiste em carregar todas as tarefas para o disco de forma sequencial para execução delas pela UCP. O

Spooling foi introduzido durante a 3ª geração de computadores e até hoje os sistemas operacionais utilizam dessa técnica.

8- A principal diferença entre uma Rede de Computadores e um Sistema Distribuído é a forma que os computadores se comportam. Em uma rede os computadores permanecem independentes e o sistema distribuído os integra para que funcionem como um só. Seus sistemas operacionais são diferentes pois o SOR faz o gerenciamento de computadores separados e o SOD faz o gerenciamento como se fosse uma máquina apenas.

9- São um meio que programas de usuário iniciam serviços do sistema operacional. É o único meio de solicitação de execução de operações protegidas fora do modo kernel.

10-

a) A entrada de dados é feita antes da execução de um JOB e a saída de dados é feita após a sua execução. Não existe interação entre o usuário e o JOB durante sua execução.

b) Permitem que o usuário dialogue com os programas durante sua execução. Podem ser monousuário ou multiusuários.

c) São usados em aplicações que utilizam sensores e interrupções e que demandam extrema agilidade.

d) Possuem alta capacidade de processamento, são projetados para realizarem diversas chamadas e também suportam programas em lote e em tempo compartilhado.

11- No modo núcleo é possível ter acesso completo de todo o hardware e suas operações e no modo usuário apenas algumas operações são executáveis. A existência desses 2 modos faz com que exista uma proteção entre os programas de usuário e o hardware.

12- As interrupções que utilizam de recursos críticos do sistema são: c, d e e.

13-

a) Todo o SO funciona como se fosse um único programa que é executado em modo núcleo e os programas de usuários são como subrotinas do SO.

b) As funções mais importantes do SO são executadas em modo núcleo e todo o restante é construído nessa base no modo usuário. Almeja-se o mínimo de código possível no modo núcleo para garantir confiabilidade.

c) O SO é organizado como uma hierarquia de camadas, onde cada nível utiliza serviços e operações dos níveis anteriores, sem saber os detalhes.

d) Possui um conjunto central de componentes e serviços adicionais podem ser adicionados por meio de módulos separados, facilitando a inserção de novas funcionalidades.

e) São SOs que utilizam diferentes abordagens para atingir desempenho, segurança e usabilidade.

14 - Nível 1 roda diretamente sobre o hardware, é eficiente e seguro independente do SO hospedeiro. Já o de Nível 2 roda sobre o sistema operacional, é mais fácil de usar, porém menos eficiente.

15 - Vantagens: VMs são independentes, facilidade de testes e aperfeiçoamento de novos SO, diminuir custos, simulação de ambientes, etc.

Desvantagens: custo adicional de processamento ao utilizar VMs, podendo passar de um aumento de 50%.

Sistemas Operacionais - Lista 2

- ① Programa é uma sequência lógica de instruções, é estático. Processo é o programa em execução, incluindo os valores atuais do PC, registradores e variáveis, é um conceito dinâmico. Nem sempre um programa equivale a apenas um processo.
- ② Código de reentrância é um programa que tem a capacidade de gerir mais de um processo, cada processo com seu conjunto de dados independente, um exemplo é o navegador web e suas abas. É vantajoso pelo fato de que não é necessário instanciar o programa inteiro a todo momento e nem apenas algumas partes.
- ③ Pois, do ponto de vista dos programas não é possível saber que um processo vai ser interrompido X vezes para ser interrompido com a execução de outros programas ou até mesmo por E/S. Por isso as suposições tem altas chances de estarem erradas, portanto, caso seja necessário um delay, este deve ser solicitado ao S.O.
- ④ ① - Inicialização do sistema: O S.O. é o 1º programa e quando é iniciado se torna um conjunto de processos, usando seu recurso de criação de processos.
② 2º Um processo em execução faz a chamada ao S.O. para que crie um processo.
③ 3º Requisição do usuário: por meio da interface gráfica ou através de comandos O usuário chama no ícone, solicitando o gerenciamento de jobs para com o recurso de processo ou diretamente no terminal.
④ → Início por jobs em lote:
O gerenciador de JOB em lote vai solicitar a criação dos processos. exemplo, arquivos .dat, o Shell que solicita.

1- Saída normal: Chamada do user (ao nível do SO) - Quando se tem programa montado/compilado, o compilador envia a chamada de a cada peça ao core, analisando uma saída normal,

- Saída por erro: Programa encontra um erro e ele mesmo faz a chamada.

2- Erro fatal: Erro não previsto pelo programa, quando uma interrupção ocorre no SO, que encerra o processo. Exemplo: divisão por 0.

3- De acordo com a Hierarquia, um processo encerra outro, de acordo com as prioridades.

4 BCP serve como uma ficha de dados dos processos, ou seja, identificadores para que o SO os reconheça. Nela é presente um ID, o estado do processo, prioridades, PC, área de armazenamento das registações e etc.

5 Um processo pode estar indefinido, ou seja, ainda ser um programa. Pronto para execução, sendo uma opção para o escalonador despachar. Em execução, ou seja, foi despachado e está em sua fatia de tempo de execução. Bloqueado, algo ocorreu e ela foi colocada em "dormência" e para ser acordada, geralmente é necessário uma interrupção, e por último Fim de processo, processo termina e seu BCP é destruído após um tempo.

6 Indefinido → Pronto para execução: criação do BCP e processo entra na lista de prontos

Pronto → Em execução: Escalonador escolhe o processo que vai ser executado e o despachante espelha o BCP para a CPU, permitindo que o processo volte do estado pausado. (Feito pelo escalonador)

Preempção: Fim da fatia de tempo ou interrupção fazem com que a CPU saia do estado atual do processo, "devolva o BCP", e o processo volta para a fila de prontos.

Bloqueado: Processo precisa que tem um evento E/S ou qualquer outro seja atendido para que seja desbloqueado.

Acabou: O evento que bloqueou foi satisfeito.

Terminou: O processo concluiu e é finalizado, seja lá qual for o tipo de término.

O escalador atua diretamente apenas na programação, despacho e gerenciamento durante no bloqueio.

⑨ processo é instância de execução do programa e cada processo possui um fluxo de controle, sua sequência de comandos, esse é chamado de thread, processos tradicionais tinham apenas um thread enquanto processos multithreads possuem vários threads que dividem os recursos e são independentes. São peças menores do processo e possuem um PC, pilha e registros próprios.

⑩ Os registradores são alocados a threads porque é a unidade real de execução na CPU. Cada thread possui seu próprio fluxo de controle e podem ter seu contexto salvo e restaurado durante um escalonamento. Um processo pode ter vários threads, e cada uma possui seu próprio conjunto lógico de registradores, mesmo que fisicamente haja apenas um físico na máquina.

⑪ Thread_yield() faz a thread ceder a CPU, pois em threads de espera de um evento não possuem interrupção por timer. Atua depois da execução das threads e do contexto que a programação insere.

⑫ Poplog por não envolver "system calls", é implementado sem depender do S.O., sendo portanto o controle do programador.

Nas demonstrações podemos citar o Bloqueio total do processo como ocorre uma chamada bloqueante: read(), write(), sleep(). Sem paralelismo real e que não ocorre sobre a existência.

3) principal vantagens: O escalonamento para a ser sem escalonamento e threads, os controlando de forma independente. Paralelismo real, comunicação e integração com o Kernel.

Desvantagens: Máximo consumo de recursos, mais lenta por conta da mudança de modo.

①.4) É a combinação, onde existe os threads de usuário, porém, eles não agrupados em threads de núcleo. Por exemplo, para cada Thread de núcleo é possível ter diversos threads de usuário.

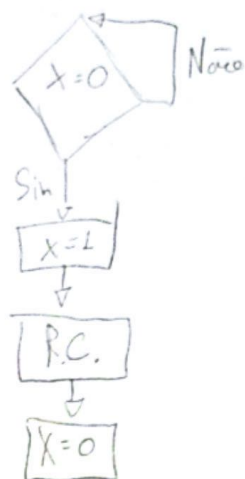
Vantagens: paralelismo real, menor overhead por conta das contagens entre threads ocorrem no modo usuário. Bloqueio eficiente.

Desvantagens: difícil implementação, sincronização delicada.

Sistemas operacionais - Lista 3

- ① É possível a comunicação por meio de uma área de armazenamento comum no disco ou memória principal. Esse acesso deve ser requerido ao S.O.
- ②
 - (a) É um bug que ocorre quando dois ou mais processos alocam sem recursos compartilhados, fazendo com que o resultado seja incorreto.
 - (b) bloco de código que envia manipulação de recursos compartilhados.
 - (c) É a relação para as condições de corrida e tem como objetivo evitar que vários de um processo leia/escreva em um recurso compartilhado ao mesmo tempo.
- ③
 - ① Dois processos não podem estar ao mesmo tempo dentro de suas R.C.s.
 - ② Nenhuma responsabilidade é feita a respeito de prioridades ou do número de UCPs.
 - ③ Nenhum processo quando fora de sua R.C. pode bloquear outros processos de entrar na R.C.
 - ④ Nenhum processo deve expor um tempo arbitrariamente longo em sua R.C.
- ④ Em sistemas com vários UCPs, desabilita interrupções em uma UCP não bloqueia os outros de acessar a memória compartilhada, portanto a saída seria travar todas as UCPs, congelando o computador o que é extremamente ineficiente.

⑤ A variável de turno é uma flag que possibilita a um processo saber se há outros processos em regiões críticas. O problema é que processo ao verificar que uma variável está sendo usada, fica com um loop fazendo as verificações, ou seja, ocupando o BCP e aumentando o tempo até a comparação mas antes de setar a flag para 1, criando a possibilidade de outros processos entrarem na região também.



⑥ A instrução TSL lê o conteúdo de uma variável de memória (adiciona valor no registrador) e em seguida coloca um valor diferente de 0 na variável lida, de forma atômica. Preservando o valor da flag, esse registrador e BCP do processador, porém a flag estará com outro valor, bloqueando outros processos.

$TSL(Reg, Flag); Reg \leftarrow Flag \text{ e } Flag \leftarrow 1$

Flag

Entrar-RC:

```

TSL(Aux, F)
CMP Aux, 0
JNZ entrar-RC
Ret
  
```

com XCHG

```

Mov Aux, 1
XCHG Aux, F
  
```

Sair-RC:

```

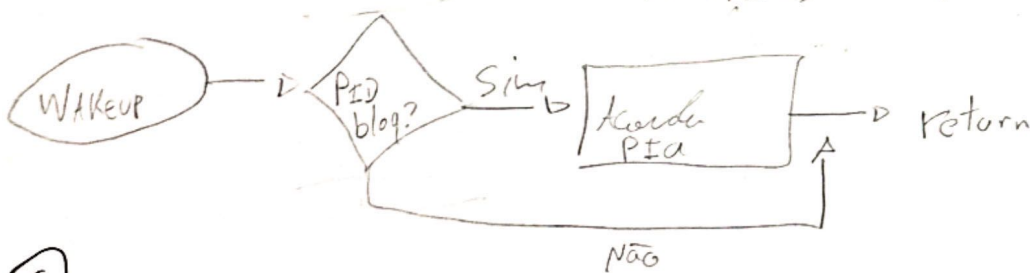
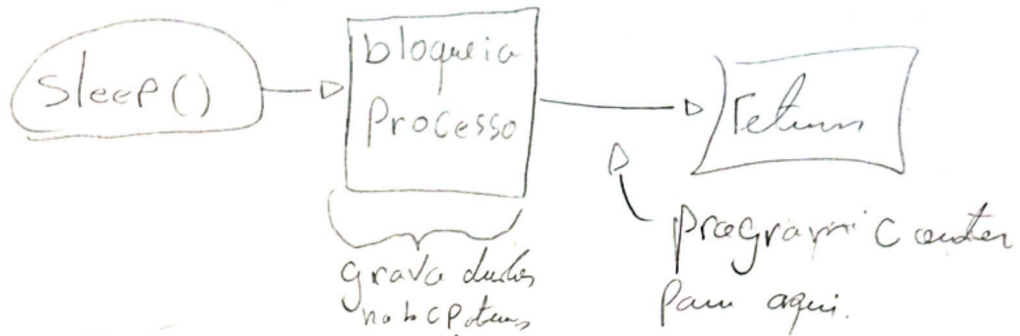
Mov F, 0
Ret
  
```

7)

Vantagens: Ao usar TSL temos uma operação atômica e sem riscos de intermissões indesejadas no meio das manuseios de variável de kernel.

Desvantagens: pode causar "bloqueio perpetuo" (deadlock) em sistemas com prioridades.

8) Sleep e Wakeup são comandos provenientes do SO, o sleep tem como função bloquear um processo, de bloquear o processo que o chama, na estrutura algorítmica faz a copia das regularidades e passa para o BCP, mudando para bloqueado ele chama o escalonador para terminar ali mesmo. O Wakeup tem o objetivo de acordar um processo que estava dormindo, ele verifica se o processo referente ao PID que ele recebe por parametro está bloqueado e se estiver, coloca o processo com pronta.



9)

```
# Define N 20
int contador
```

```
void produtor() {
```

```
    while(1) {
```

```
        Produz();
```

```
        if (contador == N) sleep();
```

```
        deposita();
```

```
        contador++;
        if (contador == 1) {
            Wakeup(consumidor);
        }
    }
```

```
}
```

```
}
```

```
}
```

```
}
```


9) Continuação

```
void Consumidor() {  
    while(1) {  
        if(contador == 0) sleep();  
        return();  
        contador --;  
        if(contador == N-1)  
            Wakeup(producer);  
        Consumi();  
    }  
}
```

Essa implementação é funcional, porém tem dois falhas. A primeira é o fato de que contador é compartilhado, portanto deveria ser manipulado utilizando variáveis de trava, tendo em vista a desabilitação dos ifs que o verificam. A segunda preocupação é se ocorrer uma interrupção entre o momento de verificar o contador e dar o sleep, pois causaria deadlock por conta da sincronização, tendo em vista que ele entraria em processo e o processo não conseguiria "acordá-lo".

10) Justamente suprir o segundo problema da questão acima. O bit é implementado no sleep e Wakeup para fazer verificação extra, ele verifica se o Wakeup tentou acordar um processo que não estava bloqueado, ou seja, caso ocorra o problema da interrupção antes de um sleep, o Wakeup vai tentar acordar esse processo, porém ele está "preso" e então o bit é colocado em alto. Ao voltar a execução o processo anteriormente "preso" estará em sleep, porém, agora há uma verificação e caso o bit seja alto, ele apenas retorna.

11) Não, a alternativa só funciona para comunicação entre dois processos (produtor/consumidor). Muitos processos implicam em condições de corrida no bit de wakeup. O risco de ruído também aumenta, tendo em vista as operações não atômicas.

Lista 36 - Sistemas Operacionais

Nome: Fernando Hiroshi Murasaki RA: 241025851

Nome: Igor dos Reis Gomes RA: 241025265

Nome: Murilo Tomaz Gonzaga RA: 241024684

1) Elas são implementadas associando uma fila de processos bloqueados a cada semáforo.

$P(s)$: Se $s > 0$ decrementa.

{ Se $s = 0$, bloqueia o processo e
coloca na fila.

$V(s)$: Se $s > 0$, retira o processo da fila
e o desbloqueia.

Se $s = 0$ incrementa.

2- Exclusão mútua: O processo faz $P(mutex)$ antes de entrar na região crítica e $V(mutex)$ ao sair. O semáforo mais adequado é o binário (inicializado em 1)

Sincronização Condicional: Um processo espera uma condição com $P(s)$ e o outro sinaliza que a condição é verdadeira com $V(s)$. O semáforo mais adequado é o geral (contador)

3- Inicia-se o semáforo com o valor N ,

- Para alocar um recurso, o processo executa $P(s)$

- Para liberar um recurso, o processo executa $V(s)$. Permitindo que até N processos acessem os recursos simultaneamente antes que o próximo seja bloqueado.

4- O mecanismo de troca de mensagens

é um mecanismo onde processos enviam e recebem dados através do SO

usando (send/receive) ao invés de variáveis compartilhadas. A sincronização

é garantida pois uma mensagem só pode ser recebida depois de ter sido enviada (se tentar receber antes, o processo espera)

5- a) Envia bloqueante e recebe bloqueante:

O remetente fica bloqueado até que a mensagem seja entregue, o receptor fica bloqueado até que a mensagem chegue, por esse comportamento, garante-se a sincronicidade.

b) Envia e recebe não bloqueante:

Ambos remetente e receptor continuam executando mesmo sem a confirmação de mensagem entregue, garante maior performance.

6- Sim, o mecanismo de troca de mensagens

pode funcionar sem problemas em um único computador. O SO gerencia a cópia da mensagem da memória do processo emissor para o receptor, garantindo a sincronicidade.

Lista de Exercícios 4 - Escalonamento de Processos

Nome: Igor dos Reis Gomes

RA: 241025265

1- A diferença é a possibilidade ou não do processo sofrer preempção durante o escalonamento. No escalonamento preemptivo existe a programação do timer que determina o tempo máximo em que o processo pode ficar na UCP, diferente do escalonamento não preemptivo, onde o processo só pode sair da UCP ao ser bloqueado ou terminado.

2-

a) $T_{\text{medio}} = (10 + 18 + 24 + 28 + 30) / 5 = 22 \text{ min}$

b) $T_{\text{medio}} = (6 + 14 + 24 + 26 + 30) / 5 = 20 \text{ min}$

c) $T_{\text{medio}} = (10 + 16 + 18 + 22 + 30) / 5 = 19.2 \text{ min}$

d) $T_{\text{medio}} = (2 + 6 + 12 + 20 + 30) / 5 = 14 \text{ min}$

3-

1ª troca:

recebe 1 quantum

$$30 - 1 = 29 \text{ restantes}$$

2ª troca:

recebe 2 quanta

$$29 - 2 = 27 \text{ restantes}$$

3ª troca:

recebe 4 quanta

$$27 - 4 = 23 \text{ restantes}$$

4ª troca:

recebe 8 quanta

$$23 - 8 = 15 \text{ restantes}$$

5ª tarefa:

recebe 16 quanta

finaliza o processo

Ocorrerão 5 tarefas e em seu último escalonamento utilizará 15 quanta (das 16 que foram disponibilizadas)

4-

$$E_1 = 0,5 \cdot 40 + 0,5 \cdot 40 = 40$$

$$E_2 = 0,5 \cdot 20 + 0,5 \cdot 40 = 30$$

$$E_3 = 0,5 \cdot 40 + 0,5 \cdot 30 = 35$$

$$E_4 = 0,5 \cdot 15 + 0,5 \cdot 35 = 25$$

A previsão será de 25 ms.

5-

$$\sum_{i=1}^n \frac{C_i}{p_i} = \frac{35}{50} + \frac{20}{100} + \frac{10}{200} + \frac{x}{250} \leq 1$$

$$\frac{x}{250} \leq 1 - 0,95$$

$$x \leq 12,5 \text{ ms}$$

O maior valor que x pode assumir para que o sistema seja escalonável é de 12,5 ms.

Lista 5 - Sistemas Operacionais

Nome: Fernando Hiroshi Murosan RA: 241025851

Nome: Igor dos Reis Gomes RA: 241025265

Nome: Murilo Tomaz Gonzaga RA: 241024684

1) Sua finalidade é gerenciar a hierarquia de memória. Ele controla quais partes da memória estão em uso e quais não estão. Aloca memória aos processos quando necessário, libera memória quando os processos terminam e trata o swapping.

2) Tipo 1: Sistemas mais simples que não fazem troca de processos (swapping) nem paginação.

Tipo 2: Sistemas com swapping e paginação (Levem e trazem processos entre memória e disco)

3) Pois nesses sistemas toda a memória é alocada à próxima tarefa, incluindo a área do próprio SO. Como não há abstração ou proteção, erros podem sobrescrever e danificar o SO.

4) a) É uma abstração da memória que define o conjunto de endereços que um processo pode usar.

b) Utiliza registradores de base e limite. Para realocação os endereços lógicos são somados ao valor de base para gerar o endereço físico. Para proteção, qualquer acesso a um endereço maior que o valor limite é erro.

5) Mapa de bits → A memória é dividida em unidades de alocação, preenchidas com 1 se estiver ocupada e 0 se livre.

Lista Encadeada → Uma lista de segmentos, onde cada elemento especifica se é um processo ou buraco.

Bitmap tendem a ocupar menos espaço quando os blocos são pequenos e numerosos.

Listas encadeadas ocupam menos espaços quando os blocos são grandes ou há poucos blocos.

c) Pior escolha

↳ 12

8

| 10 | 4 | ~~20~~ | 18 | 7 | 9 | 12 | 15 |

$$20 - 12 = 8 //$$

↳ 10

8

| 10 | 4 | 8 | ~~18~~ | 7 | 9 | 12 | 15 |

$$18 - 10 = 8 //$$

↳ 9

6

| 10 | 4 | 8 | 8 | 7 | 9 | ~~12~~ | 15 |

$$15 - 9 = 6 //$$